

SDSS classification ENG

August 28, 2026

1 Classification of Celestial Objects (SLOAN DIGITAL SKY SURVEY DR14)

2 Sloan Digital Sky Survey (SDSS)

The **Sloan Digital Sky Survey (SDSS)** represents one of the largest and most ambitious cosmic mapping projects ever undertaken.

Unlike traditional astronomical observations, which are often focused on individual objects or small regions, the SDSS was conceived to conduct a veritable “census of the Universe”: the project counts, measures, and classifies hundreds of millions of celestial objects with the goal of creating the most detailed three-dimensional map of the cosmos ever produced.

2.0.1 Primary Scientific Objectives

The fundamental purpose of the SDSS is to answer some of the most profound questions in astrophysics, foremost among them: *what does the universe look like on a large scale, and how has it evolved?*.

To do so, it focuses on the following key objectives:

- **Mapping the “Cosmic Web”:** The SDSS has revealed that galaxies are not randomly scattered across space, but are arranged in a structure resembling a sponge or a web (hence the name “cosmic web”) composed of enormous filaments of galaxies surrounding vast voids almost entirely devoid of matter. Mapping this structure is crucial for understanding evolutionary dynamics from the Big Bang to the present day.
- **Understanding Dark Matter:** More than 90% of the matter in the universe is “dark,” meaning invisible; however, by observing the distribution and motion of galaxies—which constitute visible matter—it is possible to deduce where dark matter is located and what role it played in the formation of the cosmic web.
- **Studying Galaxy Evolution:** By collecting millions of data points, the SDSS enables the study of galactic properties (morphology, star formation rate, chemical composition), helping to understand why different types exist and how they formed and evolved over time.
- **Studying the Milky Way:** The survey maps in detail the structure, kinematics, and chemical composition of our galaxy, the Milky Way, providing clues regarding its formation history.

- **Finding Quasars:** The SDSS has identified hundreds of thousands of quasars (super-luminous galactic nuclei powered by supermassive black holes), which are so distant that they allow us to probe the early universe.

2.0.2 How Does the SDSS Work?

To construct such a three-dimensional map, the SDSS employs a two-phase observational strategy, conducted primarily at the Apache Point Observatory (New Mexico) using a dedicated telescope.

1. **Imaging:** The purpose of this first phase is the creation of the 2D map that serves as a “digital catalog” for the entire universe. Specifically, four fundamental properties of celestial objects are measured with precision: *position*, *brightness*, *color*, and *morphology* (i.e., shape: point-like, such as a star or a quasar, or an extended object, such as a galaxy). The core of this system is the use of a large, advanced digital camera that utilizes **five photometric filters (u, g, r, i, z)** that isolate specific light windows, allowing the camera to capture five separate images of the same patch of sky, one for each “color.” Instead of pointing the telescope, taking a photo, and then moving (a slow process), the SDSS used a technique called “drift scanning,” in which the telescope is pointed at a strip of the sky and “frozen”—it is the Earth’s rotation itself that causes the sky to “drift” across the camera’s field of view. The digital sensors (CCDs) read the data at the exact same rate at which the sky moves, constructing a continuous, long, and uninterrupted digital image, much like a scanner passing over a document.
2. **Spectroscopy:** In this phase, the 2D map is transformed into a 3D universe by analyzing the physical nature of the objects. Specifically, the spectrum of an object is captured (by dispersing its light into its constituent colors), the analysis of which yields the following crucial results: the *nature* of the object (i.e., whether it is a star, galaxy, or quasar), its *physical properties* (chemical composition, age, temperature), and its *distance* (calculated via **redshift**, the phenomenon in which light from a distant celestial object appears shifted toward the red end of the visible spectrum—the farther away an object is, the greater its redshift); this last factor adds the z-axis to the 2D map, making it three-dimensional.

This paper aims to conduct a comparative analysis of the performance of the following classification models: Logistic Regression, K-Nearest Neighbors (KNN), Decision Tree, and Random Forest, which will be tasked with distinguishing celestial objects into the three fundamental categories: Stars (STAR), Galaxies (GALAXY), and Quasars (QSO).

3 Exploratory Data Analysis

3.1 Data Understanding

The dataset under analysis consists of ten thousand observations (rows) and 18 variables (columns), extracted from the public database of the Sloan Digital Sky Survey (SDSS Release 14 of 2017). Each row represents a single observed celestial object, while the columns contain the physical measurements and technical metadata associated with that observation. The analysis began by importing all the relevant libraries used throughout the analysis and importing the dataset to display it:

```
[1]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split , cross_val_score, \
↳StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, \
↳accuracy_score, ConfusionMatrixDisplay
from sklearn.decomposition import PCA
```

```
[2]: data=pd.read_csv('Skyserver_SQL2_27_2018 6_51_39 PM.csv')
display(data)
```

	objid	ra	dec	u	g	r \
0	1.237650e+18	183.531326	0.089693	19.47406	17.04240	15.94699
1	1.237650e+18	183.598370	0.135285	18.66280	17.21449	16.67637
2	1.237650e+18	183.680207	0.126185	19.38298	18.19169	17.47428
3	1.237650e+18	183.870529	0.049911	17.76536	16.60272	16.16116
4	1.237650e+18	183.883288	0.102557	17.55025	16.26342	16.43869
...	...	...	...	...	...	...
9995	1.237650e+18	131.316413	51.539547	18.81777	17.47053	16.91508
9996	1.237650e+18	131.306083	51.671341	18.27255	17.43849	17.07692
9997	1.237650e+18	131.552562	51.666986	18.75818	17.77784	17.51872
9998	1.237650e+18	131.477151	51.753068	18.88287	17.91068	17.53152
9999	1.237650e+18	131.665012	51.805307	19.27586	17.37829	16.30542

	i	z	run	rerun	camcol	field	specobjid	class \
0	15.50342	15.22531	752	301	4	267	3.722360e+18	STAR
1	16.48922	16.39150	752	301	4	267	3.638140e+17	STAR
2	17.08732	16.80125	752	301	4	268	3.232740e+17	GALAXY
3	15.98233	15.90438	752	301	4	269	3.722370e+18	STAR
4	16.55492	16.61326	752	301	4	269	3.722370e+18	STAR
...	...	...	...	...	...	...	...	...
9995	16.68305	16.50570	1345	301	3	161	5.033450e+17	GALAXY
9996	16.71661	16.69897	1345	301	3	162	5.033400e+17	GALAXY
9997	17.43302	17.42048	1345	301	3	162	8.222620e+18	STAR
9998	17.36284	17.13988	1345	301	3	163	5.033400e+17	GALAXY
9999	15.83548	15.50588	1345	301	3	163	5.033410e+17	GALAXY

	redshift	plate	mjd	fiberid
0	-0.000009	3306	54922	491
1	-0.000055	323	51615	541
2	0.123111	287	52023	513

3	-0.000111	3306	54922	510
4	0.000590	3306	54922	512
...	...	...	...	...
9995	0.027583	447	51877	246
9996	0.117772	447	51877	228
9997	-0.000402	7303	57013	622
9998	0.014019	447	51877	229
9999	0.118417	447	51877	233

[10000 rows x 18 columns]

3.1.1 Group 1: Identifiers

They uniquely identify celestial objects but are not linked to their physical properties.

- **objid**: stands for “Object ID” and is the unique identifier for any object identified by the camera during the imaging phase; it is associated with position data (*ra*, *dec*) and brightness data (*u*, *g*, *r*, *i*, *z*).
- **specobjid**: stands for “Spectroscopic Object ID” and is the identifier for the unique spectrum detected by the spectroscopic survey; therefore, it is associated with classification data (*STAR*, *GALAXY*, *QSO*) and distance (*redshift*).

3.1.2 Group 2: Position

These columns indicate the position of the object in the sky. * **ra**: stands for “Right Ascension” and is the *longitude* of the sky; it is measured in degrees and tells the telescope where to point in an East-West direction along the celestial equator.

- **dec**: stands for “Declination” and is the *latitude* of the sky; it is measured in degrees and tells the telescope where to point in a North-South direction.

3.1.3 Group 3: Physical Characteristics

- **redshift**: the variable that measures the Doppler effect of light; specifically, if its value is close to zero, it indicates a *star*, if it is positive, it indicates a *galaxy* (farther from us), but if its value is extremely high, it indicates a *quasar* (since they are very distant from us and also receding rapidly).
- **u, g, r, i, z**: these measure the brightness of the object and correspond respectively to: *Ultraviolet*, *Green*, *Red*, *Infrared (Near and Far Infrared)*.

3.1.4 Group 4: Classification

This constitutes the target variable of this project. * **class**: the classification of the object as *STAR*, an object within our galaxy; *GALAXY*, a system of billions of stars, gas, and dust, outside our galaxy; *QSO*, an incredibly luminous object actively “devouring” gas and stars (a supermassive black hole).

3.1.5 Group 5: Measurement Data

These describe how, when, and with which piece of equipment the measurement was performed; they are fundamental for astronomers but do not describe the celestial object itself.

- **run**: the ID of a specific sky scan.
- **rerun**: the ID of the software reprocessing version of the data.
- **camcol**: indicates which of the 6 camera columns captured the object.
- **field**: the ID of a camera shot/field.
- **plate**: the ID of the plate used by scientists to obtain the spectrum of a celestial body.
- **mjd**: the standard format used to record the exact date and time the spectrum was taken.
- **fiberid**: holes are drilled into the plate used to obtain the spectrum, to which an optical fiber is connected to carry the light to the spectrograph; this is the ID of the fiber that captured the light of a given object.

3.2 Data Cleaning and Descriptive Statistics

Before proceeding with the exploratory analysis and modeling, the dataset was first subjected to a rigorous cleaning phase. Data integrity is a fundamental prerequisite for applying any type of model: the use of “noisy”, incomplete, or improperly scaled data would inevitably lead to unreliable and misleading results.

The cleaning operations include: verifying the completeness of the dataset, removing duplicates, handling any missing values, and standardizing the data format.

Why is this phase important? 1. **Decision Quality**: if the input data are erroneous or distorted, any derived *insight* or classification performed by the model will be intrinsically flawed;

2. **Error Prevention**: identifying anomalies in this preliminary phase makes it possible to avoid cascading errors in subsequent steps, saving computational resources and debugging time;
3. **Reliability**: removing inconsistencies and noise makes the dataset more robust and reliable;
4. **Analysis Efficiency**: working on a clean dataset allows efforts to be focused exclusively on interpreting physical phenomena and optimizing algorithms, rather than on correcting errors.

3.2.1 Feature Selection

The first step involved Feature Selection, namely the removal of non-informative variables. Indeed, a high-dimensional dataset containing irrelevant data can degrade model performance and increase the risk of overfitting.

In this section, two groups of variables were eliminated: the variables belonging to Group 1, as they possess no predictive power regarding the physical nature of the object and were replaced by the progressive index of the dataset rows; and the variables belonging to Group 5, as they describe the technical configurations of the telescope and the observation date—information that constitutes merely “noise” relative to the ultimate objective of the analysis.

In this manner, the models are allowed to focus exclusively on the useful signal, improving both training speed and generalization capability.

```
[3]: data.drop(['objid', 'specobjid', 'run', 'rerun', 'camcol', 'field', 'plate', 'u',
               ↪ 'mjd', 'fiberid'], axis=1, inplace=True)
      display(data)
```

	ra	dec	u	g	r	i	z	\
0	183.531326	0.089693	19.47406	17.04240	15.94699	15.50342	15.22531	
1	183.598370	0.135285	18.66280	17.21449	16.67637	16.48922	16.39150	
2	183.680207	0.126185	19.38298	18.19169	17.47428	17.08732	16.80125	
3	183.870529	0.049911	17.76536	16.60272	16.16116	15.98233	15.90438	
4	183.883288	0.102557	17.55025	16.26342	16.43869	16.55492	16.61326	
...	...	...	...	...	...	...	...	
9995	131.316413	51.539547	18.81777	17.47053	16.91508	16.68305	16.50570	
9996	131.306083	51.671341	18.27255	17.43849	17.07692	16.71661	16.69897	
9997	131.552562	51.666986	18.75818	17.77784	17.51872	17.43302	17.42048	
9998	131.477151	51.753068	18.88287	17.91068	17.53152	17.36284	17.13988	
9999	131.665012	51.805307	19.27586	17.37829	16.30542	15.83548	15.50588	

	class	redshift
0	STAR	-0.000009
1	STAR	-0.000055
2	GALAXY	0.123111
3	STAR	-0.000111
4	STAR	0.000590
...	...	...
9995	GALAXY	0.027583
9996	GALAXY	0.117772
9997	STAR	-0.000402
9998	GALAXY	0.014019
9999	GALAXY	0.118417

[10000 rows x 9 columns]

The result of the operation is that the original dataset was reduced from 18 to 9 columns, whose corresponding variables constitute the physical core of the information.

We proceed to verify the presence of missing values and duplicates:

```
[4]: dataset_null=data.isnull()
      print(dataset_null)
```

	ra	dec	u	g	r	i	z	class	redshift
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...

```

9995  False  False  False  False  False  False  False  False  False  False
9996  False  False  False  False  False  False  False  False  False  False
9997  False  False  False  False  False  False  False  False  False  False
9998  False  False  False  False  False  False  False  False  False  False
9999  False  False  False  False  False  False  False  False  False  False

```

```
[10000 rows x 9 columns]
```

The command returns a boolean mask (where *True* indicates a missing value and *False* a present value); however, visual inspection of such a result proves difficult given the size of the dataset.

Consequently, the command below was chosen, which returns the total count of non-null values for each variable and the corresponding data type (Type), allowing us to verify that the storage format is consistent with the physical structure of the measurement.

```
[5]: data.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0    ra         10000 non-null  float64
 1   dec         10000 non-null  float64
 2    u          10000 non-null  float64
 3    g          10000 non-null  float64
 4    r          10000 non-null  float64
 5    i          10000 non-null  float64
 6    z          10000 non-null  float64
 7   class      10000 non-null  str
 8   redshift   10000 non-null  float64
dtypes: float64(8), str(1)
memory usage: 751.2 KB

```

```
[6]: print('duplicates:', data.duplicated().sum())
```

```
duplicates: 0
```

It is observed that there are neither null values nor duplicates.

Descriptive analysis is a fundamental step of EDA, as it constitutes the first true analytical stage aimed at transforming raw data into information.

The purpose of this phase is to verify the internal consistency of the dataset and the effectiveness of the preceding cleaning operations; furthermore, it also allows for acquiring a deeper understanding of the data under analysis by identifying patterns and anomalies.

```
[7]: data.describe() #for numerical type
```

```
[7]:
```

	ra	dec	u	g	r \
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	175.529987	14.836148	18.619355	17.371931	16.840963
std	47.783439	25.212207	0.828656	0.945457	1.067764
min	8.235100	-5.382632	12.988970	12.799550	12.431600
25%	157.370946	-0.539035	18.178035	16.815100	16.173333
50%	180.394514	0.404166	18.853095	17.495135	16.858770
75%	201.547279	35.649397	19.259232	18.010145	17.512675
max	260.884382	68.542265	19.599900	19.918970	24.802040

	i	z	redshift
count	10000.000000	10000.000000	10000.000000
mean	16.583579	16.422833	0.143726
std	1.141805	1.203188	0.388774
min	11.947210	11.610410	-0.004136
25%	15.853705	15.618285	0.000081
50%	16.554985	16.389945	0.042591
75%	17.258550	17.141447	0.092579
max	28.179630	22.833060	5.353854

```
[8]: data['class'].value_counts()
```

```
[8]: class
      GALAXY    4998
      STAR     4152
      QSO       850
      Name: count, dtype: int64
```

In preparation for the subsequent phase, the data were structurally separated into two distinct subsets: 1. Numerical Features Subset: a dataframe containing exclusively quantitative variables was isolated.

2. Target Variable: the categorical variable was isolated into a separate structure.

```
[9]: data_num=data.drop(['class'], axis=1)
      display(data_num)
```

	ra	dec	u	g	r	i	z \
0	183.531326	0.089693	19.47406	17.04240	15.94699	15.50342	15.22531
1	183.598370	0.135285	18.66280	17.21449	16.67637	16.48922	16.39150
2	183.680207	0.126185	19.38298	18.19169	17.47428	17.08732	16.80125
3	183.870529	0.049911	17.76536	16.60272	16.16116	15.98233	15.90438
4	183.883288	0.102557	17.55025	16.26342	16.43869	16.55492	16.61326
...	...	...	...	...	...	...	...
9995	131.316413	51.539547	18.81777	17.47053	16.91508	16.68305	16.50570
9996	131.306083	51.671341	18.27255	17.43849	17.07692	16.71661	16.69897
9997	131.552562	51.666986	18.75818	17.77784	17.51872	17.43302	17.42048
9998	131.477151	51.753068	18.88287	17.91068	17.53152	17.36284	17.13988
9999	131.665012	51.805307	19.27586	17.37829	16.30542	15.83548	15.50588

```

        redshift
0    -0.000009
1    -0.000055
2     0.123111
3    -0.000111
4     0.000590
...
9995  0.027583
9996  0.117772
9997 -0.000402
9998  0.014019
9999  0.118417

```

[10000 rows x 8 columns]

```
[10]: data_cat=data['class']
      display(data_cat)
```

```

0      STAR
1      STAR
2     GALAXY
3      STAR
4      STAR
...
9995  GALAXY
9996  GALAXY
9997   STAR
9998  GALAXY
9999  GALAXY

```

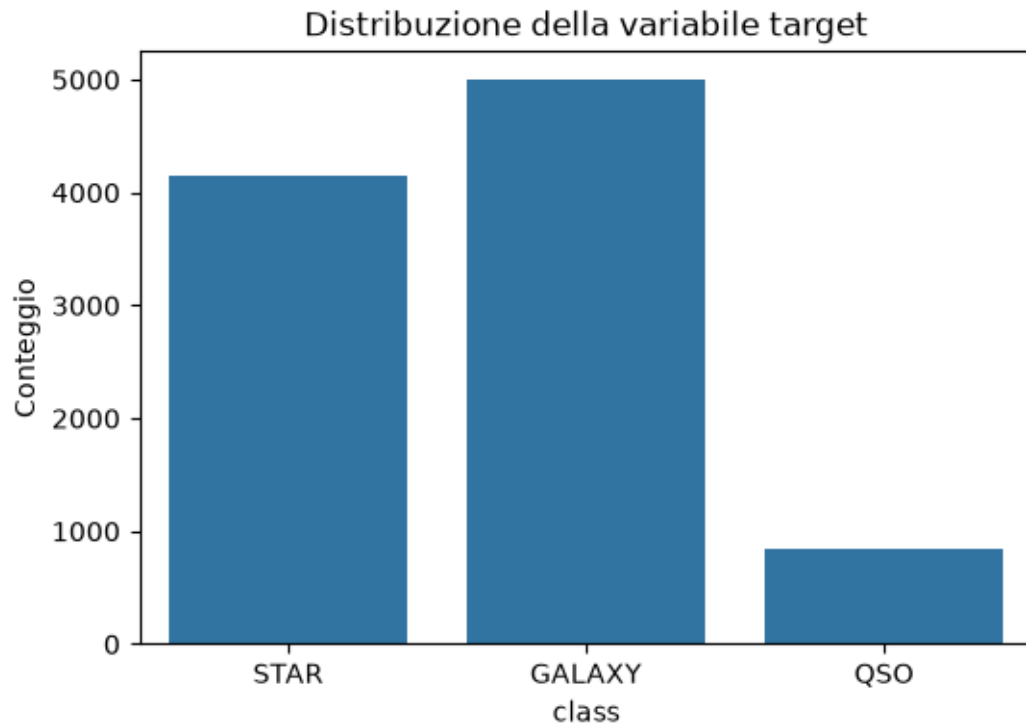
Name: class, Length: 10000, dtype: str

3.3 Univariate Analysis

The analysis of individual variables aims to identify the distributional characteristics of the data and detect any anomalies (outliers) that could negatively affect model training.

Initially, a bar plot of the target variable was constructed, which revealed the extent to which the dataset is imbalanced:

```
[11]: #target variable distribution
plt.figure(figsize=(6,4))
sns.countplot(x='class', data=data)
plt.title('Distribuzione della variabile target')
plt.ylabel('Conteggio')
plt.show()
```



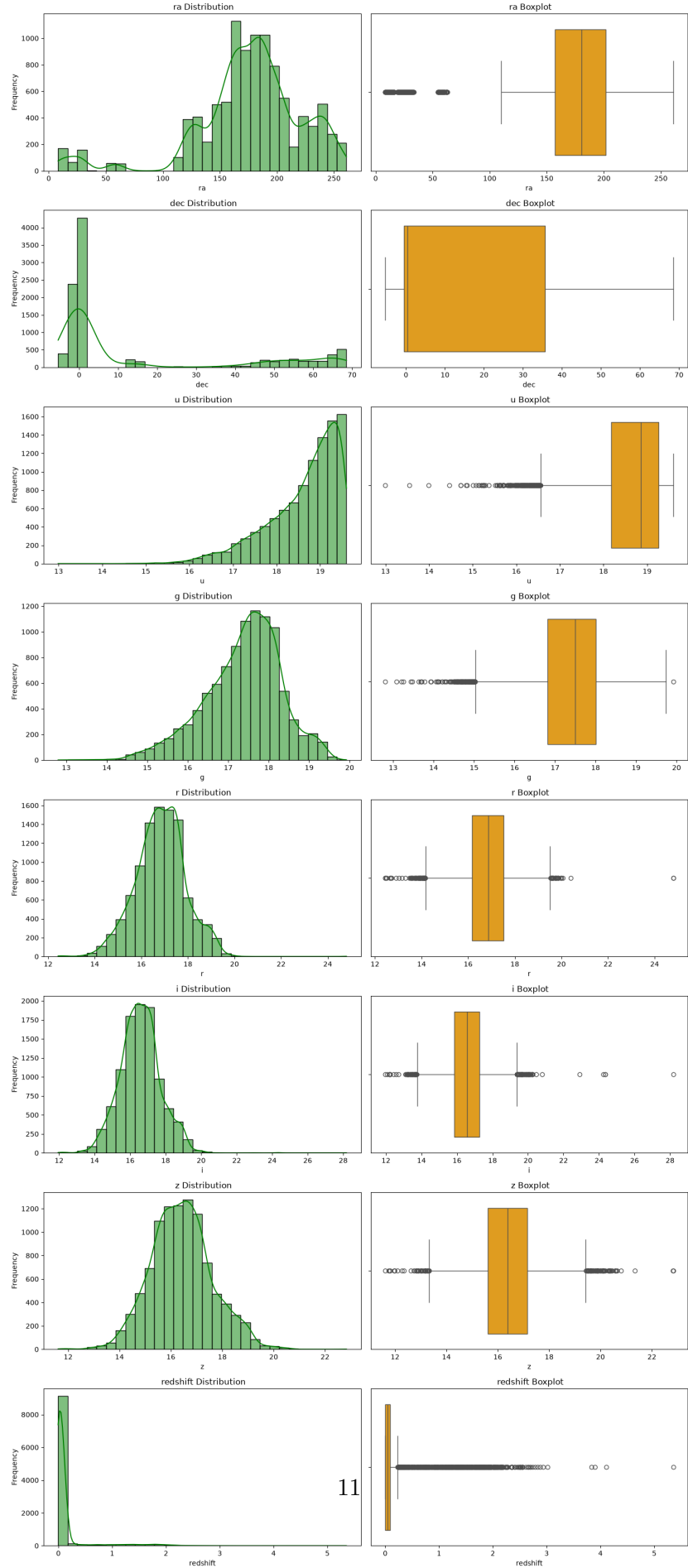
Subsequently, histograms and Box Plots were employed to visualize data distributions; in particular, the Box Plot proved helpful in detecting outliers:

```
[12]: n=8 #feature number
fig, axes=plt.subplots(n, 2, figsize=(14, 4*n))

for i, col in enumerate(data_num):
    #left side histogram
    sns.histplot(data[col], kde=True, bins=30, color='green', ax=axes[i,0])
    axes[i,0].set_title(f' {col} Distribution')
    axes[i,0].set_ylabel('Frequency')

    #right side boxplot
    sns.boxplot(x=data[col], ax=axes[i,1], color='orange')
    axes[i,1].set_title(f' {col} Boxplot')

plt.tight_layout()
plt.show()
```

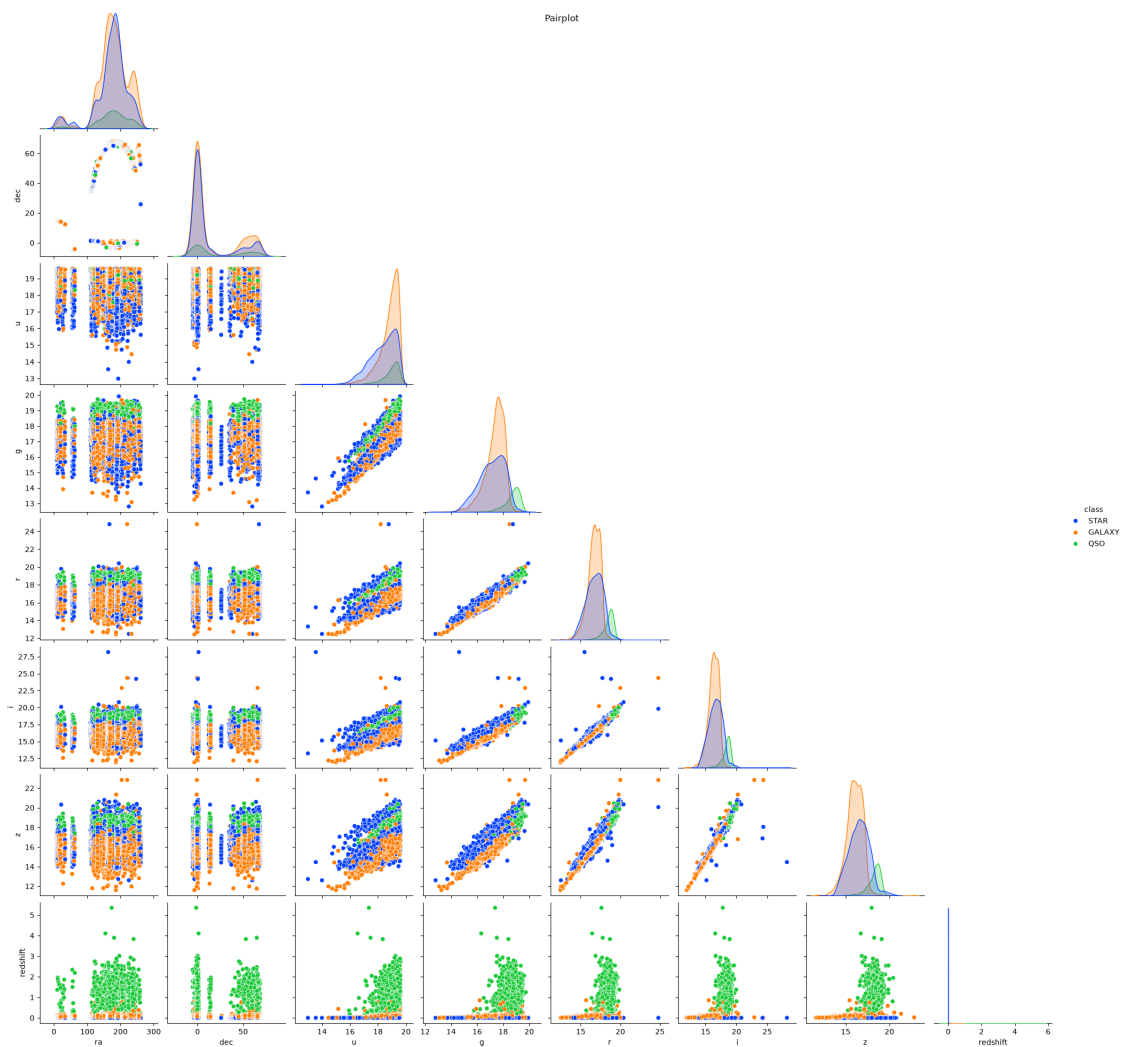


3.3.1 Bivariate Analysis

We proceeded with a Bivariate analysis to observe the interactions between the variables:

```
[13]: #pairplot
plt.figure(figsize=(15,12))
sns.pairplot(data, hue='class', palette='bright', corner=True, plot_kws=None)
plt.suptitle('Pairplot')
plt.show()
```

<Figure size 1500x1200 with 0 Axes>



Observing the top-left panel (Right Ascension vs. Declination), it can be seen that the three classes (Stars, Galaxies, and Quasars) share the same spatial distribution. This suggests that there are no

regions of the sky reserved for a single class: wherever the SDSS points the telescope, there is an equal probability of observing a star, a galaxy, or a quasar.

Therefore, the coordinates *ra* and *dec* possess no intrinsic discriminant power for the physical classification of the object. This means they could be excluded from the final predictive model, as the nature of a celestial body does not depend on its position on the celestial sphere.

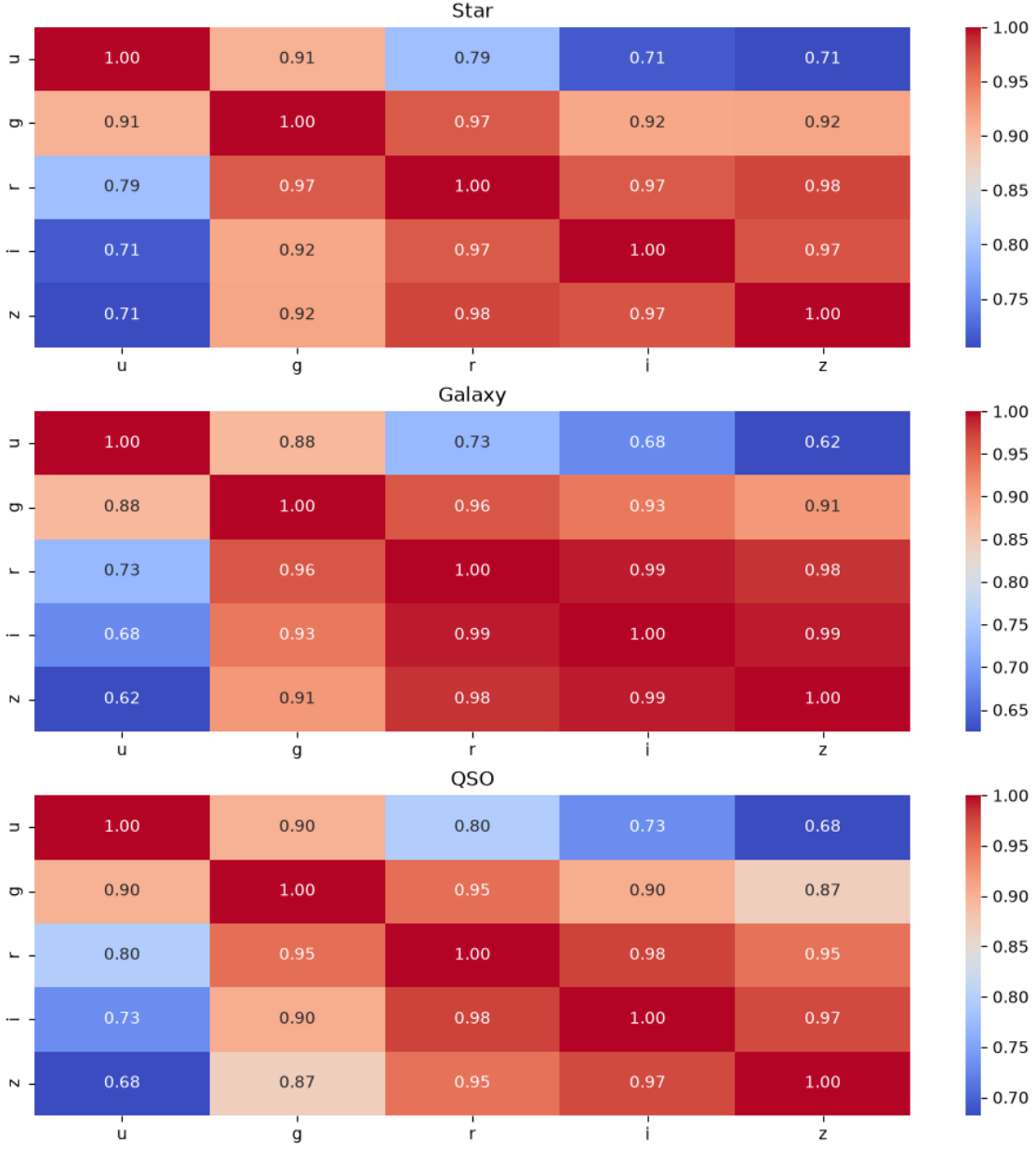
Observing the bottom row of the plot (Redshift on the X-axis vs. other features) clarifies the nature of the outliers detected during the univariate analysis; in fact, they are not measurement errors, but correspond exclusively to the values assumed by the QSO class. Furthermore, it can be observed that Quasars are geometrically well separated from the other two classes along the redshift axis, suggesting that even a simple linear model would suffice to classify quasars. Conversely, it is observed that Stars and Galaxies are distributed across the same redshift range and remain indistinguishable.

However, observing the magnitude analysis reveals the subtle difference between Stars and Galaxies (impossible to discern through redshift alone). It is precisely this difference that enables non-linear models such as Random Forest to classify Stars and Galaxies—a task in which a simple linear model would fail due to the overlap.

The magnitude plots show strong collinearity, as the data points align along a well-defined diagonal. A strong positive correlation is thus observed, indicating that if an object is bright in one band, it tends to be bright in the others as well.

Therefore, to conclude: - Redshift acts as a primary filter to clearly separate Quasars; - The relationships between spectral indices act as a secondary filter to disentangle the overlap between Stars (point-like and coherent spectrum) and Galaxies (more complex and dispersed spectrum).

```
[14]: fig, axes= plt.subplots(nrows=3, ncols=1, figsize=(12,12))
ax = sns.heatmap(data[data['class']=='STAR'][['u', 'g', 'r', 'i', 'z']].corr(),
    ↪ax = axes[0], cmap='coolwarm', annot=True, fmt='.2f')
ax.set_title('Star')
ax = sns.heatmap(data[data['class']=='GALAXY'][['u', 'g', 'r', 'i', 'z']].
    ↪corr(), ax = axes[1], cmap='coolwarm', annot=True, fmt='.2f')
ax.set_title('Galaxy')
ax = sns.heatmap(data[data['class']=='QSO'][['u', 'g', 'r', 'i', 'z']].corr(),
    ↪ax = axes[2], cmap='coolwarm', annot=True, fmt='.2f')
ax = ax.set_title('QSO')
```



It is observed that the correlation (from the Pearson coefficients) is approximately the same for each class; therefore, the different spectral bands behave in the same manner across the various classes. This introduces information redundancy, which will be addressed in the following section.

Subsequently, the Data Encoding phase was conducted, a crucial preprocessing step consisting of converting categorical (textual) variables into a numerical format.

This transformation is an essential technical prerequisite for model training. Most Machine Learning algorithms operate on matrices and therefore require exclusively numerical inputs.

Specifically for the dataset under analysis, the target variable “class” was transformed, mapping its

textual labels to unique integer values using the *Label Encoding* technique (GALAXY=0, QSO=1, STAR=2).

```
[15]: le=LabelEncoder()  
      data['class']=le.fit_transform(data['class'])
```

It should be noted that Label Encoding was preferred over One-Hot Encoding due to the technical specifications of the Scikit-learn library.

The implementation of classification algorithms, such as Random Forest and Decision Tree, requires that the target vector y passed to the *.fit()* method be a one-dimensional array.

3.3.2 Feature Engineering

3.3.3 PCA

The previous analysis highlighted strong multicollinearity among the five photometric bands (u , g , r , i , z); therefore, it was decided to intervene to reduce dataset redundancy without losing informational content.

Specifically, PCA (Principal Component Analysis) was applied with the objective of obtaining a lower-dimensional representation of the data. The technique was applied selectively and exclusively to the five photometric bands, leading to the extraction of two mutually orthogonal principal components, **PC1** and **PC2**. Mathematically, each component is a linear combination of the original features: the coefficients of this combination, the **loadings**, indicate how much each original variable contributes to the construction of the new component.

In addition to resolving the issue of multicollinearity, this dimensionality reduction offers a significant advantage in terms of **computational efficiency**: by working with a smaller number of variables, computational complexity is reduced, and the time required for model training is substantially decreased.

```
[16]: #from the pairplot we observe that we can delete ra and dec  
      data=data.drop(['ra', 'dec'], axis=1)
```

```
[17]: pca=PCA(n_components=2)  
      u_g_r_i_z= pca.fit_transform(data[['u','g','r','i','z']])  
  
      #update dataframe  
      data=pd.concat((data,pd.DataFrame(u_g_r_i_z)), axis=1)  
      data.rename({0:'PCA_1', 1:'PCA_2'}, axis=1, inplace=True)  
      data.drop(['u','g','r','i','z'], axis=1, inplace=True)  
      data.head()
```

```
[17]:
```

	class	redshift	PCA_1	PCA_2
0	2	-0.000009	-1.507202	1.377293
1	2	-0.000055	-0.195758	0.028410
2	0	0.123111	1.297604	0.590023
3	2	-0.000111	-1.446117	-0.566685
4	2	0.000590	-0.849271	-1.287505

The split of the updated data was then performed:

```
[18]: X=data.drop('class', axis=1)
      Y=data['class']
      display(X)
      print('='*60)
      display(Y)
```

	redshift	PCA_1	PCA_2
0	-0.000009	-1.507202	1.377293
1	-0.000055	-0.195758	0.028410
2	0.123111	1.297604	0.590023
3	-0.000111	-1.446117	-0.566685
4	0.000590	-0.849271	-1.287505
...	...	...	...
9995	0.027583	0.222959	0.134301
9996	0.117772	0.259171	-0.415333
9997	-0.000402	1.480725	-0.388717
9998	0.014019	1.392088	-0.117004
9999	0.118417	-0.936205	1.113215

[10000 rows x 3 columns]

```
=====
0      2
1      2
2      0
3      2
4      2
..
9995   0
9996   0
9997   2
9998   0
9999   0
```

Name: class, Length: 10000, dtype: int64

The dataset was partitioned into two disjoint subsets: the **Training Set**, used for model training, and the **Test Set**, used for the final performance evaluation.

The split was performed according to the following criterion: 70% Training Set and 30% Test Set.

Furthermore, the *stratify* parameter was utilized to ensure that the proportion of the three target classes (Stars, Galaxies, and Quasars) remained identical in both subsets, preserving statistical representativeness even for the less frequent classes.

```
[19]: #70/30
```

```
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.3,
↪random_state=42, stratify=Y)
```

3.3.4 Feature Scaling

Next, we proceeded with the *scaling* of the variables; in particular, the **Standard Scaler** was utilized, which transforms the distribution of each feature so that it has: a Mean equal to zero and a Standard Deviation equal to 1.

It should be noted that the Scaler was applied exclusively to the Training Set, which allows *Data Leakage* to be avoided.

```
[20]: scaler=StandardScaler()
x_train_scaled=scaler.fit_transform(x_train)
x_test_scaled=scaler.transform(x_test)
```

3.4 Model Training

As anticipated in the introductory phase, the central objective of the project is the selection of the best classifier for distinguishing celestial objects into Stars, Galaxies, and Quasars. To address this multiclass classification problem, the following four modeling techniques were adopted:

3.4.1 Logistic Regression

Logistic regression represents the standard parametric approach for modeling the posterior probability that an observation belongs to a specific class, given the values of the predictors. Unlike linear regression, which is unsuitable for qualitative response variables as it could generate inadmissible probability estimates (negative or greater than unity), logistic regression adopts a specific link function that constrains the modeled output to the valid probability interval.

According to the frequentist approach described in the literature, model coefficients are not estimated by minimizing the residual sum of squares (RSS), but rather through the Maximum Likelihood Estimation (MLE) method. This iterative process identifies the parameters that maximize the joint probability of observing the sample data actually collected (Likelihood Function).

Since the present study analyzes a response variable with three categories, the standard binary model was extended to Multinomial Logistic Regression. In this context, the generalization of the logistic function is achieved through the application of the Softmax function. This algorithm normalizes the outputs of the linear model, transforming them into probability vectors whose sum equals unity, thereby ensuring a valid probability distribution across all K possible classes. The predicted class corresponds to the one associated with the highest estimated probability.

3.4.2 K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric classification method distinguished by the absence of explicit assumptions regarding the functional form of the underlying data distribution. Instead of estimating global parameters, the algorithm operates according to a “memory-based” logic, determining classification exclusively based on the local structure of the data in the feature space.

The decision-making process is structured by identifying, for each new observation, a set of K points in the training set closest to it, calculated according to Euclidean distance. The estimated

probability of class membership is derived from the relative frequency of that class within the identified neighborhood.

Crucial to this process is the choice of the parameter K (set here by default to 5), which directly governs the flexibility of the model: * A small value of K results in a highly flexible model with irregular decision boundaries, minimizing bias but exposing the model to high variance (risk of overfitting).

- A large value of K produces smoother and more stable decision boundaries, reducing variance at the expense of increased bias (risk of underfitting).

3.4.3 Decision Trees

Decision Trees operate through a hierarchical segmentation strategy of the predictor space. The methodological approach is defined as *top-down* and *greedy*: starting from the entire dataset, the algorithm proceeds by performing successive binary splits (recursive binary splitting) with the objective of maximizing the homogeneity of the resulting nodes.

The selection of the split variable and the optimal split point is not based on classification error, which is considered insufficiently sensitive, but on the use of statistical purity measures such as the Gini index or Entropy. These metrics quantify the degree of disorder or total variance among the classes within a specific node; low values of these indices indicate that the node consists predominantly of observations belonging to the same class.

To mitigate the natural tendency of trees to overfit the complexity of the data, the methodology includes a cost-complexity pruning phase. This technique penalizes the size of the tree, allowing for the selection of a more parsimonious structure that marginally sacrifices accuracy on the training set to achieve superior generalization capability on unseen data.

3.4.4 Random Forest

Random Forests constitute an *ensemble learning* technique designed to refine the Bagging (Bootstrap Aggregation) method, reducing prediction variance without altering bias. The primary limitation of simple Bagging lies in the correlation among the generated trees: in the presence of strong predictors, most bootstrap trees will tend to use the same variables for the initial splits, rendering them structurally similar.

Random Forest achieves the decorrelation of estimators. During the construction of each decision tree, at each node split, the algorithm is constrained to consider only a random subset of the available predictors. This stochastic restriction prevents dominant variables from monopolizing the structure of the forest, ensuring greater diversity among individual trees.

Aggregating the predictions of decorrelated trees (via majority voting) produces a final estimate characterized by significantly lower variance compared to individual trees or standard Bagging, resulting in greater robustness and predictive accuracy.

3.4.5 Performance Evaluation Metrics

To quantify the predictive capability of the implemented classification models, it is necessary to evaluate the discrepancy between the predicted classes $\hat{y}$ and the actual classes y . The evaluation is based on the construction of the **Confusion Matrix**, from which the main performance metrics are derived: *Accuracy*, *Precision*, *Recall*, and *F1-Score*.

1. Confusion Matrix

A confusion matrix is a $K \times K$ table that compares the classifications predicted by the model with the classes observed in the test set. Specifically, the matrix elements are defined as **True Positives (TP)**, positive observations correctly classified as positive; **True Negatives (TN)**, negative observations correctly classified as negative; **False Positives (FP)**, negative observations incorrectly classified as positive (Type I Error); and **False Negatives (FN)**, positive observations incorrectly classified as negative (Type II Error).

2. Accuracy

Accuracy represents the most intuitive performance metric and indicates the ratio between the number of correctly classified observations and the total number of observations. However, relying exclusively on accuracy can be misleading, particularly in the presence of imbalanced datasets (as in the case under examination).

3. Precision

Precision is the ratio of correctly predicted positive observations (TP) to the total predicted positive observations (TP+FP); it therefore quantifies the reliability of the classifier in predicting the positive class. Consequently, high precision implies a low false positive rate.

3. Recall (Sensitivity)

Recall, also known as *Sensitivity* or *True Positive Rate*, is the ratio of correctly predicted positive observations (TP) to the total actual positive observations (TP+FN); specifically, it measures the model's ability to identify all positive instances present in the dataset. Precision and Recall are particularly relevant in scenarios characterized by class imbalance (as in this case).

4. F1-Score

The F1-Score is the (weighted) harmonic mean of Precision and Recall. Unlike the arithmetic mean, the harmonic mean heavily penalizes extreme values. Therefore, the F1-Score is a robust metric for imbalanced datasets, as it attains a high value only when both Precision and Recall are simultaneously high.

To evaluate model performance on the test set, **Stratified Cross-Validation** was employed given the imbalanced nature of the dataset. Stratification ensures that the minority class is equally represented in each testing fold.

```
[21]: # Define the models to be compared in a dictionary
from sklearn.multiclass import OneVsRestClassifier
models= {
    'Logistic Regression (Softmax)': LogisticRegression(max_iter=5000,
↳random_state=42),
    'Logistic Regression (OvR)':
↳OneVsRestClassifier(LogisticRegression(solver='lbfgs', max_iter=5000,
↳class_weight='balanced', random_state=42)),
    'K-Nearest Neighbors (KNN)': KNeighborsClassifier(),
    'Decision Tree':DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}
```

```

#now training and testing the models with cross-validation
accuracies={}
conf_matrix={}
class_names=['GALAXY','QSO', 'STAR']

print('\n'+ '='*60)
print('Starting Cross-Validation and Test')
print('='*60)

cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for name, model in models.items():
    #Cross-Validation
    cv_scores=cross_val_score(model, x_train_scaled, y_train, cv=cv,
    ↪scoring='accuracy')
    mean_acc=cv_scores.mean()
    std_acc=cv_scores.std()

    model.fit(x_train_scaled,y_train) #training

    y_hat=model.predict(x_test_scaled) #prediction

    #calcolo l'accuratezza
    acc=accuracy_score(y_test, y_hat)

    accuracies[name]=acc
    conf_matrix[name]=confusion_matrix(y_test, y_hat)

    print(f'Modello: {name}')
    print(f'Accuratezza CV (5-fold): {mean_acc:.4f} (+/- {std_acc:.4f})')
    print(f'Accuratezza Test Set: {acc:.4f}')
    print('-'*60)
    print(classification_report(y_test, y_hat, target_names=class_names))

#Graph and final comparison of the models
plt.figure(figsize=(18,10))
plt.bar(accuracies.keys(), accuracies.values(), color=['pink', 'yellow',
    ↪'green', 'orange', 'purple'])
plt.title('Confronto Accuratezza Modelli')
plt.ylabel('Accuratezza')
plt.ylim(0.7, 1.0)
plt.show()

```

=====

Starting Cross-Validation and Test

=====

Modello: Logistic Regression (Softmax)

Accuratezza CV (5-fold): 0.9610 (+/- 0.0019)

Accuratezza Test Set: 0.9600

	precision	recall	f1-score	support
GALAXY	0.98	0.94	0.96	1499
QSO	0.98	0.87	0.92	255
STAR	0.94	1.00	0.97	1246
accuracy			0.96	3000
macro avg	0.96	0.94	0.95	3000
weighted avg	0.96	0.96	0.96	3000

Modello: Logistic Regression (OvR)

Accuratezza CV (5-fold): 0.9211 (+/- 0.0028)

Accuratezza Test Set: 0.9197

	precision	recall	f1-score	support
GALAXY	0.95	0.88	0.92	1499
QSO	0.93	0.97	0.95	255
STAR	0.89	0.95	0.92	1246
accuracy			0.92	3000
macro avg	0.92	0.94	0.93	3000
weighted avg	0.92	0.92	0.92	3000

Modello: K-Nearest Neighbors (KNN)

Accuratezza CV (5-fold): 0.9473 (+/- 0.0032)

Accuratezza Test Set: 0.9540

	precision	recall	f1-score	support
GALAXY	0.98	0.92	0.95	1499
QSO	0.99	0.95	0.97	255
STAR	0.92	0.99	0.95	1246
accuracy			0.95	3000
macro avg	0.96	0.95	0.96	3000
weighted avg	0.96	0.95	0.95	3000

Modello: Decision Tree

Accuratezza CV (5-fold): 0.9850 (+/- 0.0028)

Accuratezza Test Set: 0.9880

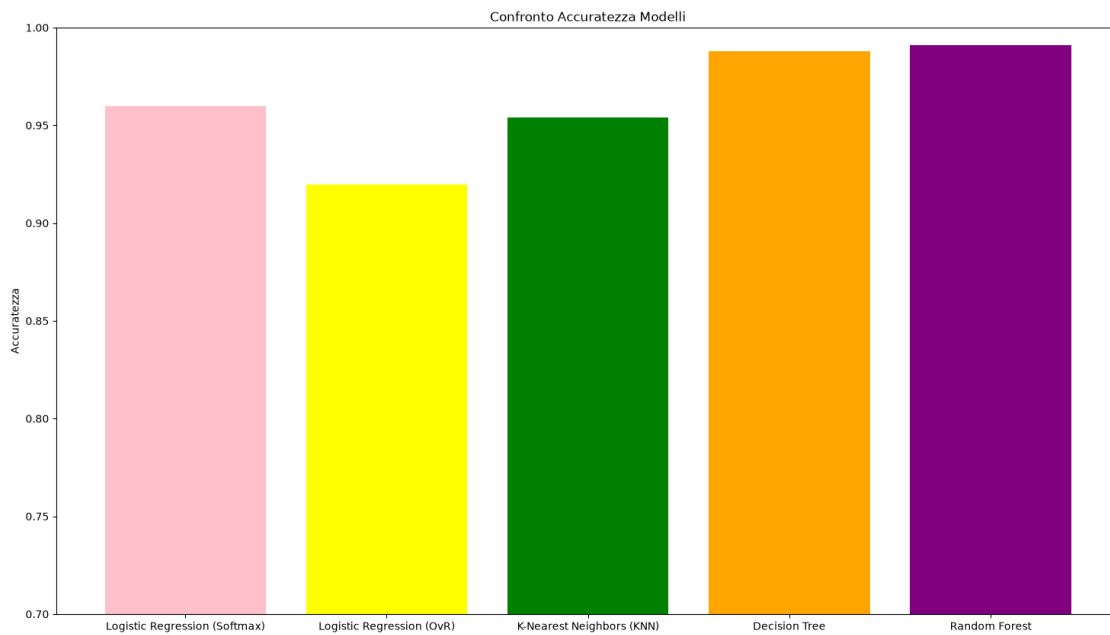
	precision	recall	f1-score	support
GALAXY	0.99	0.99	0.99	1499
QSO	0.95	0.93	0.94	255
STAR	1.00	1.00	1.00	1246
accuracy			0.99	3000
macro avg	0.98	0.97	0.98	3000
weighted avg	0.99	0.99	0.99	3000

Modello: Random Forest

Accuratezza CV (5-fold): 0.9880 (+/- 0.0027)

Accuratezza Test Set: 0.9910

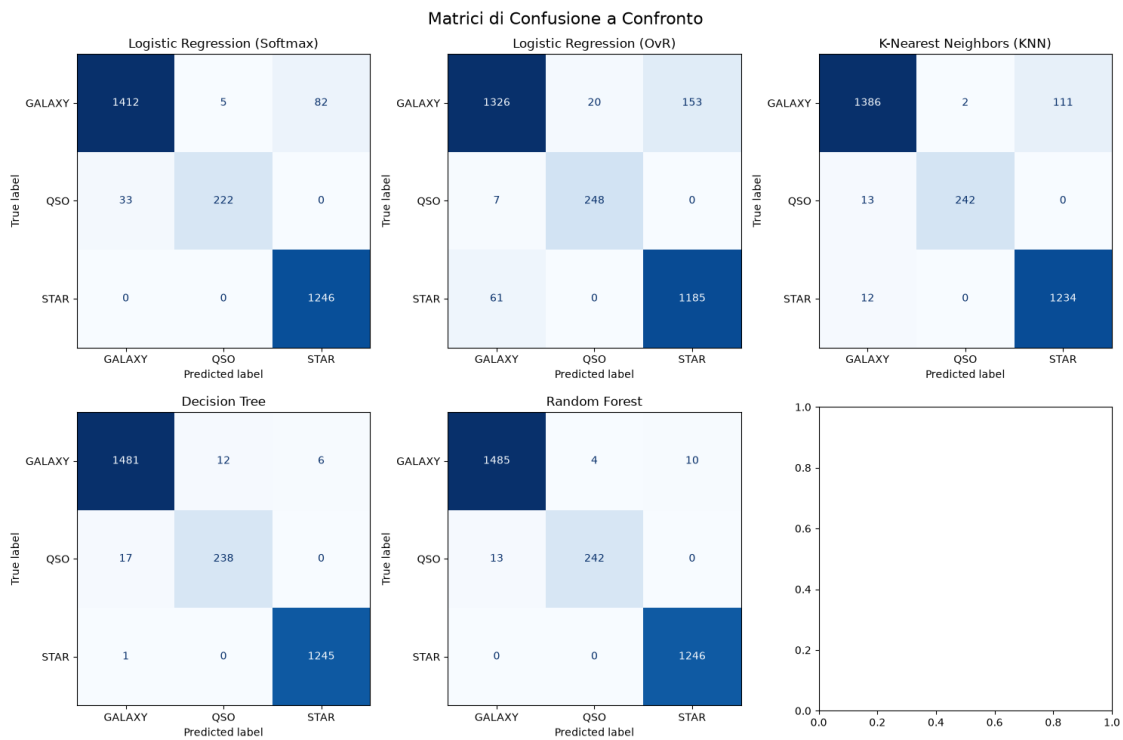
	precision	recall	f1-score	support
GALAXY	0.99	0.99	0.99	1499
QSO	0.98	0.95	0.97	255
STAR	0.99	1.00	1.00	1246
accuracy			0.99	3000
macro avg	0.99	0.98	0.98	3000
weighted avg	0.99	0.99	0.99	3000



```
[22]: #confusion matrix comparison
fig, axes=plt.subplots(2,3, figsize=(15,10))
axes=axes.flatten()

for i, (name,cm) in enumerate(conf_matrix.items()):
    disp=ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
    disp.plot(ax=axes[i], cmap='Blues', colorbar=False)
    axes[i].set_title(name)

plt.suptitle('Matrici di Confusione a Confronto', fontsize=16)
plt.tight_layout()
plt.show()
```



Logistic Regression provided a solid baseline, with an overall accuracy on the test set of 96%. However, the analysis of the QSO class reveals the structural limitations of the model: with a Recall of 0.87, logistic regression missed the identification of 13% of the QSO objects; as can also be observed from the confusion matrix, 33 Quasar observations were erroneously classified as Galaxies.

This result confirms what had already been observed in the Pair Plot, namely that the relationship between the features of Quasars and those of Galaxies is not purely linear; the bias introduced by the linearity assumption penalized the sensitivity on the minority class.

K-Nearest Neighbors recorded the lowest overall accuracy. Although the non-parametric nature of KNN allowed for better capturing the local structure of QSOs—as can be observed from the Recall

rising to 0.95 and from the confusion matrix where only 13 Quasars were incorrectly classified as Galaxies—the model failed to correctly classify Stars and Galaxies. This is due to the fact that KNN is sensitive to noise in boundary regions between dense classes, where the local majority voting rule can generate false positives; as observed from the confusion matrix, 111 Galaxies were confused with Stars (the boundary points of the Galaxies were likely absorbed by the local density of the Stars).

The transition from linear and distance-based models to the **Decision Tree** marked a clear improvement in predictive capabilities. The most notable result is the perfect performance on the Star class, as can be seen from both the F1-Score and the confusion matrix. This indicates that stars possess physical characteristics that clearly distinguish them from Galaxies and Quasars, given that the *recursive binary splitting* algorithm successfully identified sharp splitting rules. The Decision Tree also demonstrated a degree of robustness in handling class imbalance; as can be seen from the confusion matrix, only 17 Quasars were incorrectly classified as Galaxies; although this number is slightly higher than that obtained with KNN, the tree offers superior overall balance.

Random Forest is the best performer overall, confirming the superiority of the ensemble approach—namely, confirming that averaging multiple trees reduced variance without increasing bias.

3.5 Multinomial Logistic Regression: Softmax vs. One-vs-Rest

Although the **scikit-learn** library uses the **Softmax** approach as the default standard for multiclass classification problems, a comparative experiment was conducted by implementing the **One-vs-Rest (OvR)** strategy in order to evaluate whether decomposing the problem into independent binary classifiers might offer advantages compared to joint modeling.

With the Softmax approach, the model considers all classes simultaneously; conceptually, the model elects one of the classes as a reference category (*baseline*) and estimates the logarithm of the odds ratios (*log-odds*) of all other classes relative to it. The distinctive feature of this approach is that the estimated probabilities for the K classes are constrained to be mutually exclusive and to sum to unity. This means that during training, an increase in the probability assigned to one class results in a decrease in the probabilities of the others, allowing the model to capture competitive relationships among the different categories.

With the One-vs-Rest approach, the multiclass problem is split into K independent binary classification problems. In this case, three distinct classifiers are trained, each specialized in recognizing a single class relative to the rest of the dataset:

- **Model 1 (Star vs. Rest)**: distinguishes Stars from everything that is not a Star;
- **Model 2 (Galaxy vs. Rest)**: distinguishes Galaxies from everything that is not a Galaxy;
- **Model 3 (Quasar vs. Rest)**: distinguishes Quasars from everything that is not a Quasar;

To classify a new observation, it is fed into all three models simultaneously; each model produces a probability output, and the algorithm assigns the observation to the class that achieved the highest probability score.

Experimental analysis shows that the Softmax approach is globally superior to OvR, recording a Test set accuracy of 96.00% compared to 91.97% for OvR. Although OvR is globally less accurate, it exhibited a significantly higher Recall for the minority class (Quasars), correctly identifying 97%

of Quasars; however, this gain came at the expense of precision on the majority classes. This likely occurred because, as observed in the bottom row of the pair plot, Quasars constitute a separate and distinct cluster from the rest of the data; thus, the independent binary classifier *Quasar vs. Rest* was able to effectively isolate this class without suffering interference from the other categories.

Conversely, the plot highlights significant overlap between the clusters of Stars and Galaxies; therefore, the *Star vs. Rest* and *Galaxy vs. Rest* classifiers struggle in this region. The Softmax approach, on the other hand, excels at this separation because it transforms this classification problem into a direct competition among categories; by imposing that the probabilities be mutually exclusive (summing to unity), the model penalizes ambiguities in overlap regions, forcing the algorithm to delineate sharper decision boundaries. In particular, the model learns not only “what resembles a Star” but, above all, “what distinguishes a Star from a Galaxy”.

3.6 Statistical Inference

In order not to limit the analysis solely to the model’s predictive capability, but rather to understand which variables drive classification decisions, an inferential analysis was conducted on the β coefficients estimated by the Multinomial Logistic Regression (Softmax) model.

3.6.1 Bootstrap

Non-parametric Bootstrap allows for quantifying the uncertainty of sample estimates without relying on stringent assumptions regarding the theoretical distribution of the data. The procedure, performed over $B = 1000$ iterations, was structured into the following phases:

1. **Sampling (Resampling):** starting from the original training dataset of size n , B bootstrap samples were generated by drawing n observations with replacement; that is, in each individual bootstrap sample, some original observations may appear multiple times while others may be omitted;
2. **Parameter Estimation:** for each bootstrap sample b , the Logistic Regression model was re-trained, yielding a new set of coefficient estimates. This step simulates the variability that would be obtained by repeatedly drawing samples from the actual population;
3. **Standard Error Calculation:** the standard error (SE) of each coefficient was estimated by computing the standard deviation of the bootstrap coefficient distribution. This estimate was subsequently utilized to derive the z-scores and corresponding p-values, testing the null hypothesis that the predictor has no effect on the response variable ($H_0 : \beta_j = 0$).

```
[23]: from scipy.stats import norm
      from sklearn.utils import resample
      from sklearn.base import clone

      n_iterations = 1000      # number of bootstrap iterations
      log_reg = models['Logistic Regression (Softmax)']

      # loop bootstrap
      bootstrap_coefs = []
```

```

for i in range(n_iterations):
    X_res, y_res = resample(x_train, y_train, stratify=y_train, random_state=i)

    # Clone the model to avoid altering the original one
    model_boot = clone(log_reg)
    model_boot.fit(X_res, y_res)

    bootstrap_coefs.append(model_boot.coef_)

# array numpy 3D (n_iterations, n_classes, n_features)
bootstrap_coefs = np.array(bootstrap_coefs)

# Compute the statistics of the bootstrap coefficients
std_errors = bootstrap_coefs.std(axis=0)

original_coefs = log_reg.coef_ # Extract the coefficients of the original model

#test
z_scores = original_coefs / (std_errors + 1e-10)

p_values = (1 - norm.cdf(np.abs(z_scores))) * 2

# Construct the table for the final comparison
frames = []

for i, class_name in enumerate(class_names):

    df_class = pd.DataFrame({
        'Coef': original_coefs[i],
        'P-Value': p_values[i]
    }, index=x_train.columns)

    # Add significance asterisks: * <0.05, ** <0.01, *** <0.001
    df_class['Sig.'] = df_class['P-Value'].apply(
        lambda p: '***' if p < 0.001 else ('**' if p < 0.01 else ('*' if p < 0.
↪05 else ''))
    )

    frames.append(df_class)

final_report = pd.concat(frames, axis=1, keys=class_names)

```

```

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
pd.set_option('display.float_format', '{:.4f}'.format)

print('\n'+ '='*100)
print('Parameter estimates of the Logistic Regression (Softmax) model with_
their respective p-values and significance levels:')
print('='*100)

print(final_report)

```

```

=====
=====
Parameter estimates of the Logistic Regression (Softmax) model with their
respective p-values and significance levels:
=====
=====

```

	GALAXY			QSO			STAR		
	Coef	P-Value	Sig.	Coef	P-Value	Sig.	Coef	P-Value	Sig.
redshift	9.8372	0.0000	***	12.8441	0.0000	***	-22.6812	0.0000	***
PCA_1	-0.5723	0.0000	***	0.5904	0.0000	***	-0.0181	0.3225	
PCA_2	0.5611	0.0000	***	-0.5560	0.0000	***	-0.0051	0.9134	

The table reveals the following observations:

1. The Dominant Role of Redshift

The redshift variable emerges as the most powerful and statistically significant discriminant across all three classes, albeit with differing effects: for *GALAXY* and *QSO*, the coefficients are strongly positive—a result consistent with astrophysical theory, as Galaxies and Quasars are distant extragalactic objects; therefore, an increase in redshift drastically increases the log-probability of an object belonging to either of these two classes. For *STAR*, the coefficient is strongly negative, indicating that elevated redshift values severely penalize the probability that an object is a Star. In other words, stars are local objects that exhibit low redshift; as redshift increases, the probability of the object being a Star drops toward zero.

2. Principal Components as a Discriminant Between Galaxies and Quasars

For *GALAXY* and *QSO*, both PCA components are highly significant; in particular, a sign inversion is observed. Because both Galaxies and Quasars exhibit high redshift values, this variable alone is insufficient to distinguish them; the model therefore utilizes spectral information (PCA) to separate these two classes.

3. The Non-Significance of Principal Components for the STAR Class

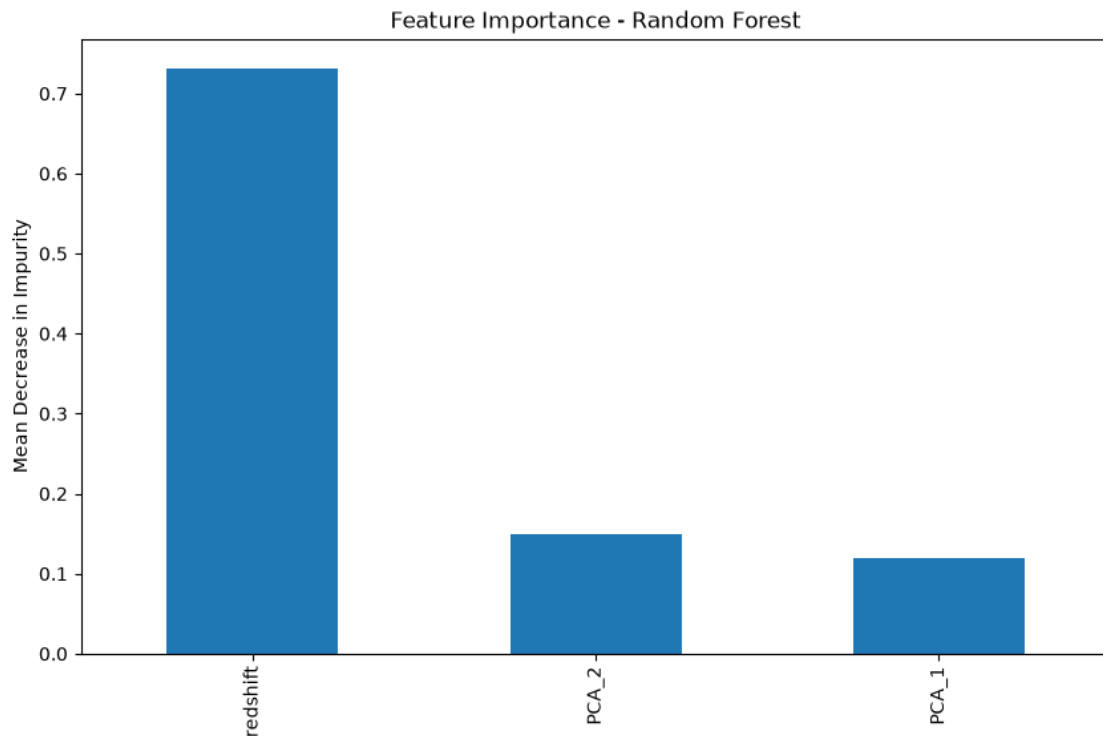
It is observed that the PCA components are not statistically significant for the *STAR* class; furthermore, the coefficients are approximately zero. This suggests that, to classify a Star, the model relies almost exclusively on redshift.

This analysis confirms the physical consistency of the model: Redshift performs an initial macroscopic stratification, segregating local sources (Stars) from cosmological ones. Subsequently, the principal components assume a crucial role in classifying distant objects (Galaxies and Quasars) thanks to the subtle differences in their spectral profiles.

3.6.2 Feature Importance - Random Forest

Unlike Logistic Regression, which utilizes the β coefficients, Random Forest estimates feature importance by computing the Mean Decrease in Impurity.

```
[24]: rf_model = models['Random Forest']  
      imp=rf_model.feature_importances_  
      feature_names=x_train.columns  
  
      rf_featimp=pd.Series(imp, index= feature_names).sort_values(ascending=False)  
  
      # Plot  
      plt.figure(figsize=(10, 6))  
      rf_featimp.plot.bar()  
      plt.title('Feature Importance - Random Forest')  
      plt.ylabel('Mean Decrease in Impurity')  
      plt.show()
```



As highlighted by the bar chart, the redshift variable overwhelmingly dominates all other features,

confirming its role as the primary predictor for classification.

This result is entirely consistent with the underlying physics of the problem.

Random Forest learned that redshift acts as a “primary separator”: it enables a clear partition of the dataset into two macro-groups (local, low-redshift objects such as Stars, and cosmological, high-redshift objects such as Galaxies and Quasars).

The concordance between the findings of Random Forest and Logistic Regression confirms that redshift is an intrinsic and robust property of the astrophysical data: distance serves as the fundamental discriminating feature of the observable Universe.

Elena Cascone